NEXUS - Manual Técnico (Tron Grid Edition)

Programación II — Documentación y Preparación para el Examen H3

Proyecto base: `NexusTronLegacy` | Versión del Sistema: v2.97

I. Arquitectura y Programación Orientada a Objetos (POO)

Propósito general: NEXUS es una simulación multidimensional de consola basada en el universo de *Tron*. El flujo principal inicializa una sesión de usuario (`UsuarioGrid`), digitalizándolo en un sistema de coordenadas de cuadrícula (`SistemaTronGrid`) para gestionar su ciclo de vida mientras navega por sectores, encuentra nodos de memoria, combate contra entidades de red y busca completar directivas primarias.

Estructura del proyecto y responsabilidades de clases

- `VisualesTron`: Clase estática utilitaria para renderizar encabezados y efectos de glitch en consola.
- `NodoMemoria`: Modelo de bloque de datos con identificador, tipo (`TipoNodo`) y capacidad de procesamiento.
- `SubsistemaMemoria`: Inventario lineal (`List<NodoMemoria>`) con límite de 8 bloques para gestionar sub-rutinas.
- `EntidadRed`: Clase base abstracta que modela identidad y ciclos de integridad (vida).
- `UsuarioGrid`: Hereda de `EntidadRed`; añade energía, disco de protección, contadores CPU y memoria.
- `SondaCentinela` / `RectificadorCLU`: Entidades enemigas con comportamientos específicos.
- `DirectivaDirectorio` y `SistemaTronGrid`: Control de objetivos de misión y orquestación del núcleo principal.

II. Fundamentos de Clases y Objetos

Instanciación: Una clase es la plantilla (ej. `NodoMemoria`) y un objeto es su representación creada en memoria mediante `new` (ej. `_usuarioActual = new UsuarioGrid(idCadete);`).

Encapsulamiento: Se protegen variables internas mediante modificadores (`private`, `protected`), exponiendo alteraciones controladas a través de métodos como `ModificarEnergia` con validación `Math.Clamp`.

III. Herencia, Polimorfismo e Interfaces

Herencia y Clases Abstractas: `EntidadRed` define atributos comunes para evitar duplicación de código. Su método abstracto `EjecutarInteraccion` obliga a las clases derivadas a implementar sus propias reglas de combate.

Métodos Virtuales y Override: El método `RecibirEfectoDerez` es `virtual` en la clase base y se sobrescribe (`override`) en `UsuarioGrid` para priorizar la absorción de daño a través del `DiscoProteccion`:

```
public override void RecibirEfectoDerez(int cantidad) {
    if (DiscoProteccion > 0) {
        int absorbido = Math.Min(DiscoProteccion, cantidad);
        DiscoProteccion -= absorbido;
        cantidad -= absorbido;
    }
    if (cantidad > 0) { base.RecibirEfectoDerez(cantidad); }
}
```

IV. Colecciones y LINQ

Listas Genéricas y Expresiones Lambda: El inventario utiliza `List<NodoMemoria>`. Mediante LINQ y expresiones lambda se realizan consultas declarativas eficientes:

```
public bool TieneDepurador() => _bloques.Exists(x => x.Tipo == TipoNodo.Depurador);
```

Esto resulta considerablemente más limpio y legible que un bucle `foreach` iterativo manual con variables booleanas auxiliares.

V. Análisis del Funcionamiento y Trazabilidad

Flujo de Navegación: Al seleccionar desplazar moto de luz, se actualizan las coordenadas `(_gridX, _gridY)`, se descuenta energía y se activa `EscanearAreaSector()`, evaluando probabilidades aleatorias para eventos de combate o nodos.

Predicción de Código: Si se ejecuta `_usuarioActual.ModificarEnergia(-120);`, el uso interno de `Math.Clamp` evita valores negativos, fijando la energía en `0` de manera segura.

VI. Preparación para el Examen y Prompt Espejo

Preguntas Clave de Nivel Universitario

1. ¿Por qué `EntidadRed` es abstracta? Porque representa un concepto genérico cuyos atributos aplican a cualquier actor, pero no tiene sentido instanciar una entidad sin definir si es usuario o enemigo específico.

2. ¿Qué ocurre si se omite `base.RecibirEfectoDerez` en la clase derivada? El daño superaría el escudo pero la integridad del ciclo nunca disminuiría, volviendo inmortal al usuario ante ataques masivos.

VII. Resultado del Prompt Espejo (Diagnóstico de Aprendizaje)

- **Fortalezas:** Rigurosidad metodológica al seguir la ruta operativa del proyecto, verificación de herramientas y extracción correcta de código fuente.
- **Diagnóstico Hito 3:** Dominio comprobado en POO, constructores, encapsulamiento, herencia, polimorfismo, colecciones y LINQ.
- **Plan de Estudio de Refuerzo:** 1. Jerarquía de constructores con herencia y palabra `base(...)`. 2. Sobrecarga de operadores y métodos estáticos/dinámicos. 3. Consultas LINQ avanzadas con `OrderByDescending`. 4. Excepciones personalizadas. 5. Contratos rigurosos con Interfaces (`IDepurable`).

Nota de integración: Este diagnóstico consolida el cierre metodológico del proyecto formativo de Programación II.